ОТЧЁТ по Лабораторной работе №4

Выполнил: Тоц Л. А.
Группа: ИВТ-2

---

1. ВЫПОЛНЕННЫЕ ТРЕБОВАНИЯ

1.1. Основное задание

Маршрут `/size2json`

- Принимает PNG изображения через `multipart/form-data`
- Возвращает JSON с размерами: `{"width": 1920, "height": 1080}`
- Проверка формата файла (только PNG)
- При неверном формате: `{"result": "invalid filetype"}`

Маршрут `/login`

- Возвращает логин автора: `{"author": "1153307"}`

JSON ответы

- Все данные возвращаются в формате JSON
- Правильные заголовки `Content-Type: application/json`

---

2. ВЫПОЛНЕННЫЕ УСЛОЖНЕНИЯ

2.1. Реализация фронтенда на Vue.js 3

Использованные технологии:

- **Backend:** Flask (Python)
- **Frontend:** Vue.js 3 (CDN)
- **CSS:** Кастомные стили с градиентами и анимациями

Особенности реализации:

- Реактивный интерфейс

- Компонентная структура
- Computed свойства для отображения имён файлов
- Асинхронные методы (async/await)

2.2. Асинхронная отправка данных

Реализация:

- Отправка файлов без перезагрузки страницы (Fetch API)
- Обработка ответов в реальном времени
- Индикация загрузки (`loading` state)
- Обработка ошибок

Маршруты:

- `POST /showPicture` — загрузка и отображение изображения
- `GET /lastUpload` — получение последнего загруженного файла
- `POST /clearUploads` — очистка папки uploads

2.3. Сохранение состояния

Клиентская часть (localStorage):

```
localStorage.setItem('lastUpload', JSON.stringify(data));
```

Серверная часть (JSON файл):

```
STATE_FILE = 'last_upload.json'
# Сохранение состояния между перезапусками сервера
```

Функционал:

- Приложение помнит последний загруженный файл
- Данные сохраняются после перезагрузки страницы
- Данные сохраняются после перезапуска сервера

2.4. Отображение изображений

Возможности:

- Предпросмотр выбранного файла (имя файла)
- Отображение загруженного изображения на странице

- Показ последнего загруженного файла
- Корректная обработка MIME-типов
- Автоматическое определение формата (PNG, JPEG, GIF, и т.д.)

---

3. ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ

3.1. Структура проекта

```
ЛР4/
├── main.py                 # Flask приложение
├── last_upload.json        # Сохранение состояния
├── templates/
│   └── index.html          # Vue.js шаблон
├── static/
│   ├── app.js              # Vue.js логика
│   └── style.css           # Современные стили
└── uploads/                # Загруженные файлы
```

3.2. Основные маршруты Flask

Маршрут	Метод	Описание
`/`	GET	Главная страница с формой
`/login`	GET	Возврат логина автора
`/size2json`	POST	Получение размеров PNG
`/showPicture`	POST	Загрузка и сохранение изображения
`/lastUpload`	GET	Получение последнего файла
`/clearUploads`	POST	Очистка папки uploads
`/uploads/<filename>`	GET	Доступ к загруженным файлам

3.3. Валидация изображений

Поддерживаемые форматы (PIL):

- PNG, JPEG, JPG, GIF, BMP, WEBP, ICO
- TIFF, PSD, RAW, и другие (всего 40+ форматов)

Проверка:

```python
def is_allowed_image(file):
    img = Image.open(file.stream)
    img.verify()
    return img.format in PIL_FORMATS
```

3.4. Современный UI/UX

CSS особенности:

- Градиентный фон (фиолетовый)
- Анимации при наведении (transform, box-shadow)
- Кастомные кнопки с градиентами
- Адаптивный дизайн
- Плавные переходы (transition)
- Тени для глубины (box-shadow)

Vue.js реактивность:

- Отслеживание выбора файлов
- Динамическое обновление интерфейса
- Условный рендеринг (v-if)
- Привязка классов (:class)

---

4. ДОПОЛНИТЕЛЬНЫЕ ВОЗМОЖНОСТИ

4.1. Реализовано сверх требований

Открытие JSON в новой вкладке

- Чистый JSON без HTML обёртки
- Blob + URL.createObjectURL()

Красивый интерфейс

- Современные кнопки с градиентами
- Анимации hover эффектов
- Индикация загрузки

Обработка ошибок

- Try/catch блоки

- Пользовательские сообщения об ошибках
- Валидация на клиенте и сервере

Очистка загруженных файлов

- Кнопка "Очистить папку uploads"
- Подтверждение действия
- Удаление файлов с сервера

4.2. Оптимизации

- Использование `io.BytesIO` для работы с памятью
- Сброс позиции потока (`file.stream.seek(0)`)
- Генерация уникальных имён файлов (UUID)
- Автоматическое создание папок (`os.makedirs`)

---

5. ИСПОЛЬЗОВАННЫЕ ТЕХНОЛОГИИ

Backend

- **Python 3.13**
- **Flask** — веб-фреймворк
- **Pillow (PIL)** — обработка изображений
- **UUID** — генерация уникальных имён

Frontend

- **Vue.js 3** — реактивный фреймворк
- **Fetch API** — асинхронные запросы
- **LocalStorage** — клиентское хранилище
- **HTML5** — структура
- **CSS3** — стили и анимации

Дополнительно

- **JSON** — формат обмена данными
- **Multipart/form-data** — загрузка файлов
- **Blob API** — работа с бинарными данными

---

6. ПРИМЕРЫ РАБОТЫ

6.1. Запрос размеров PNG

Request:

```
POST /size2json
Content-Type: multipart/form-data

image: screenshot.png (1920x1080)
```

Response:

```
{
  "width": 1920,
  "height": 1080
}
```

6.2. Загрузка изображения

Request:

```
POST /showPicture
Content-Type: multipart/form-data

image: photo.jpg
```

Response:

```
{
  "image_url": "/uploads/abc123def456.jpg",
  "filename": "abc123def456.jpg",
  "ext": "jpg"
}
```

6.3. Получение последнего файла

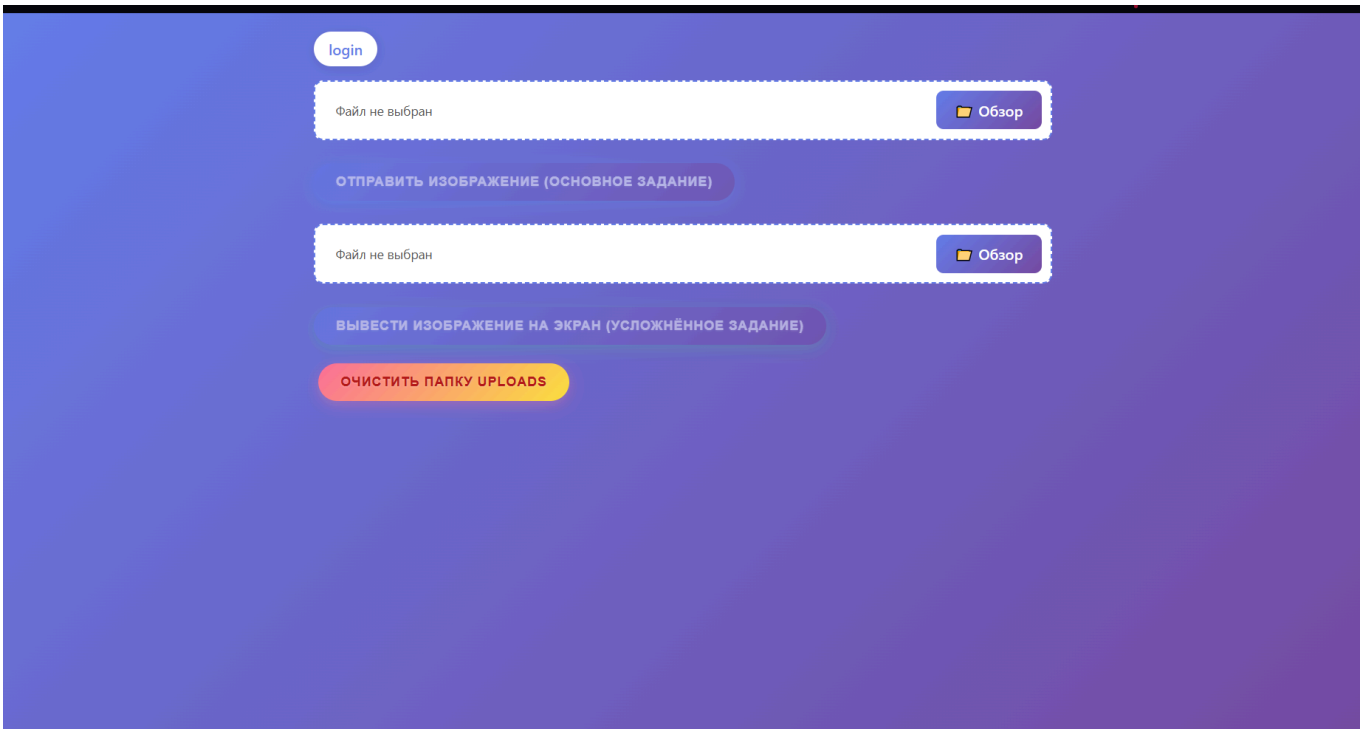
Request:

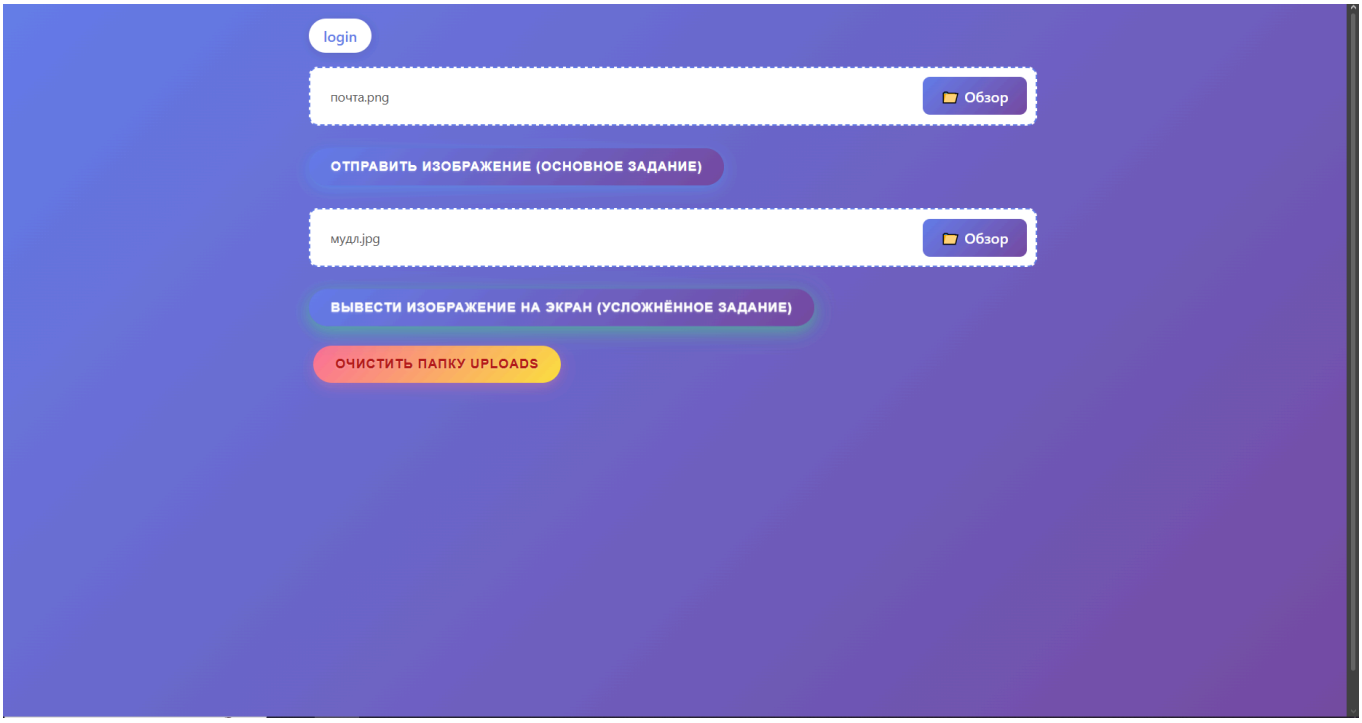
```
GET /lastUpload
```

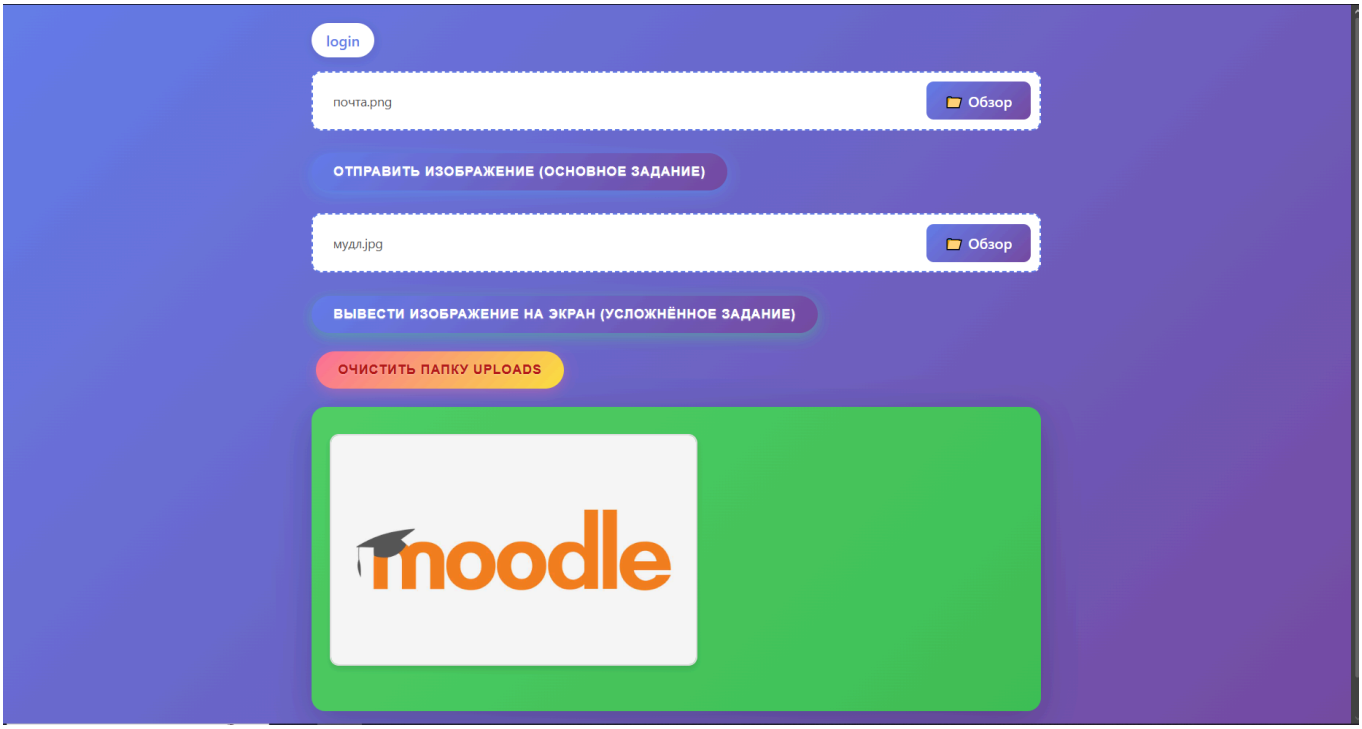
Response:

```
{
  "image_url": "/uploads/abc123def456.jpg",
  "filename": "abc123def456.jpg",
  "ext": "jpg"
}
```

Скриншоты работы сайта







---

7. ВЫВОДЫ

Выполнено:

Основное задание (100%)
Усложнение 1: Vue.js фреймворк
Усложнение 2: Асинхронная отправка
Усложнение 3: Сохранение состояния
Усложнение 4: Отображение изображений

Достигнутые результаты:

- Полностью рабочее веб-приложение
- Современный отзывчивый интерфейс
- Корректная обработка ошибок
- Сохранение данных между сессиями
- Чистый и поддерживаемый код
- Соответствие всем требованиям задания

Технические особенности:

- Использован современный стек (Flask + Vue.js 3)
- Реализована реактивность без сборки (CDN)

- Применены best practices (async/await, error handling)
- Оптимизирована работа с файлами (stream, seek)